

Report

"Inlämningsuppgift 1: Containerbaserad webbapplikation — från inner loop till Azure Container Apps"

Author: Claes Fransson

Date: May 13, 2026

Repository: <https://github.com/Claes1981/inlamningsuppgift1>

1. Introduction

This report documents the work completed for Assignment 1 of the Cloud Developer course (Molnapplikationer Fördjupning). The assignment requires building a containerized .NET MVC web application (CloudSoft Recruitment Portal) and deploying it to Azure Container Apps with CI/CD.

The CloudSoft Recruitment Portal is a web application that allows candidates to browse and apply for job postings, and administrators to create, manage, publish, and close job postings. The application uses Azure CosmosDB as its data store and implements role-based access control with two user roles: Administrator and Candidate.

The Docker code is not yet committed to Git since it doesn't yet fully work.

2. Agile Working Method and Inner Loop (Delmoment 1)

2.1 User Stories

Three user stories have been created and documented in `doc/user_stories/`:

US-1: Browse Job Postings - As a candidate, I want to browse available job postings so that I can find opportunities that match my skills and interests. - Acceptance criteria defined for viewing published jobs, displaying job details, filtering by status, and handling empty results.

US-2: Apply for Job - As a candidate, I want to apply for a job posting so that I can express my interest in a position. - Acceptance criteria defined for application form, validation, confirmation, and duplicate prevention.

US-3: Manage Job Postings - As an administrator, I want to create, edit, publish, and close job postings so that I can manage the recruitment process. - Acceptance criteria defined for CRUD operations, role-based access control, status transitions, and audit logging.

2.2 Inner Loop Description

The inner loop (development cycle) has so far mainly followed this workflow:

1. **Edit:** Tell Qwen3.6-27B model via Opencode harness and Llama.cpp llama-server what I

wish accomplished. Give it access to the assignment description document and the online course exercises.

2. **Build:** Run `dotnet build src/CloudSoft.Web/CloudSoft.Web.csproj` to compile the solution
 3. **Test:** Run `dotnet run --project src/CloudSoft.Web/CloudSoft.Web.csproj` to test locally
 4. **Containerize:** Run `podman compose up --build` to test in containers
-

3. Containerization and Local Development Environment (Delmoment 2)

3.1 Dockerfile

A multi-stage Dockerfile has been created with three stages:

Stage 1 - Build (`mcr.microsoft.com/dotnet/sdk:10.0`): - Copies solution file (`CloudSoft.slnx`) and source code - Runs `dotnet restore` to fetch NuGet packages - Runs `dotnet build` in Release configuration

Stage 2 - Publish (inherits from build): - Runs `dotnet publish` with `/p:UseAppHost=false` to produce a framework-dependent deployment

Stage 3 - Runtime (`mcr.microsoft.com/dotnet/aspnet:10.0`): - Copies published output from the publish stage - Exposes port 8080 - Sets `ASPNETCORE_HTTP_PORTS=8080` - Entry point: `dotnet CloudSoft.Web.dll`

Key design decisions: - The `aspnet:10.0` image is a chiseled (minimal) image that does not include `curl` or `adduser`. Therefore, no HEALTHCHECK instruction and no non-root user creation are included in the Dockerfile. - Dependency restore happens inside the SDK container (not on the host) to ensure reproducible builds. - The `.dockerignore` file excludes `**/obj/`, `**/bin/`, and other unnecessary files from the build context.

3.2 Docker Compose

A `docker-compose.yml` file orchestrates two services:

webapp service: - Builds from the local Dockerfile - Maps port 8080 - Configures environment variables for CosmosDB connection - Depends on the cosmosdb service (waits for healthy status)

cosmosdb service: - Uses `mcr.microsoft.com/cosmosdb/linux/azure-cosmos-emulator:latest` - Maps port 8081 - Includes health check with `-fk` flag for self-signed certificate - Has 60-second start period and 10 retry attempts - Allocated 3GB memory limit

3.3 Container Testing

The containerized application has been tested successfully: - `podman build --format docker -t cloudsoft-recruitment .` builds without errors - `podman compose up` starts both services

- The CosmosDB emulator reaches healthy status - The webapp container starts and listens on port 8080 - A health endpoint at `/health` returns a healthy status response

4. Authentication, Authorization, and Data Layer (Delmoment 3)

4.1 Authentication

Cookie-based authentication is implemented in `AccountController` : - Two hardcoded user accounts for demonstration purposes: - Administrator: `admin` / `admin123` - Candidate: `candidate` / `candidate123` - Claims-based identity with `ClaimTypes.Name` and `ClaimTypes.Role` - Login, logout, and access denied pages implemented - Cookies configured with `HttpOnly` , `SecurePolicy.Always` , and `SameSiteMode.Strict`

4.2 Authorization

Role-based authorization is implemented: - `JobPostingsController` is decorated with `[Authorize(Roles = "Administrator")]` — only administrators can manage job postings - `HomeController` allows both authenticated and anonymous access for browsing published jobs - The authentication and authorization middleware is properly ordered in the middleware pipeline

4.3 Data Layer — CosmosDB Repository

A generic repository pattern is implemented:

Domain Layer (`CloudSoft.Domain`):- `JobPosting` entity with properties: `Id`, `Title`, `Description`, `Location`, `Salary`, `Status`, `CreatedAt`, `UpdatedAt` - `JobApplication` entity with properties: `Id`, `JobPostingId`, `CandidateName`, `Email`, `ResumeUrl`, `AppliedAt` - `JobPostingStatus` enum: `Draft`, `Published`, `Closed` - `ApplicationStatus` enum: `Pending`, `Accepted`, `Rejected` - `IRepository<T>` interface with CRUD operations

Data Layer (`CloudSoft.Data`):- `CosmosRepository<T>` implements `IRepository<T>` using the Azure Cosmos SDK - Supports: `GetByIdAsync` , `GetAllAsync` , `AddAsync` , `UpdateAsync` , `DeleteAsync` - Uses SQL API query mode with `QueryDefinition`

Services Layer (`CloudSoft.Services`):- `IJobPostingService` / `JobPostingService` provides business logic - CRUD operations plus `PublishAsync` and `CloseAsync` for status transitions - `GetPublishedAsync` returns only published job postings

Dependency Injection (configured in `Program.cs`):- `CosmosClient` registered as singleton - `IRepository<JobPosting>` registered as singleton (factory creates database/container references) - `IJobPostingService` registered as scoped

5. CI/CD Pipeline and Azure Deployment (Delmoment 4)

5.1 Status: NOT YET IMPLEMENTED

The following work remains to be completed:

GitHub Actions Workflow — A CI/CD pipeline needs to be created that: - Triggers on push to `main` branch and on pull requests - Builds the .NET solution and runs `dotnet build` - Runs `dotnet test` if tests exist - Builds the container image using `docker build` or `podman build` - Pushes the image to Azure Container Registry (ACR) - Deploys to Azure Container Apps

Azure Container Registry (ACR) — An ACR instance needs to be provisioned to store container images.

Azure Container Apps — The application needs to be deployed to Azure Container Apps with: - Environment variables for CosmosDB connection string - Ingress configuration for HTTP/HTTPS traffic - Scaling rules (if applicable)

6. Infrastructure as Code — Bicep (Delmoment 4)

6.1 Status: NOT YET IMPLEMENTED

A Bicep template needs to be created that provisions: - Azure Resource Group - Azure Container Registry (ACR) - Azure CosmosDB account (with SQL API) - Azure Container Apps environment - Azure Container App (referencing ACR image) - Managed Identity for secure access between resources - Role assignments for CosmosDB access

The Bicep template should use parameters for environment-specific values and outputs for resource URIs.

7. Verification of Deployed Solution (Delmoment 5)

7.1 Status: NOT YET IMPLEMENTED

Once the Azure infrastructure is provisioned and the CI/CD pipeline is working, the following verification steps need to be performed: - Confirm the application is accessible via the Container Apps endpoint - Test login as Administrator and Candidate - Verify CRUD operations for job postings - Verify role-based access control (candidate cannot access admin functions) - Capture screenshots of the running application

8. Security Considerations

8.1 Implemented

- Cookie authentication with `HttpOnly` and `SecurePolicy.Always` flags
- `SameSiteMode.Strict` to prevent CSRF attacks
- Role-based authorization with `[Authorize(Roles = "Administrator")]`
- HTTPS redirection enabled in the middleware pipeline

8.2 Remaining Work

- Hardcoded credentials in `AccountController` needs replacement with a proper identity provider (e.g. ASP.NET Core Identity) in production.
- Secrets management: connection strings must be stored in Azure Key Vault or GitHub Actions secrets, never committed to the repository.
- Managed Identity should be used for CosmosDB access in production
- CORS and rate limiting should be considered for production deployment.

9. AI Assistant Usage

Throughout this assignment, AI coding assistant (Opencode - Llama.cpp - Qwen3.6-27B) have been used to: - Generate initial project structure and boilerplate code - Create Dockerfile and docker-compose.yml configurations - Implement the CosmosDB repository pattern - Debug containerization issues (e.g., chiseled image limitations, certificate handling) - Help write documentation and this report

The AI assistant suggestions were reviewed, tested, and adapted by the developer. Almost all code has been verified to compile and run correctly.

10. Summary of Completed and Remaining Work

Deliverable	Status	Notes
User stories (Delmoment 1)	✓ Complete	3 user stories documented
Inner loop description (Delmoment 1)	✓ Complete	Edit-build-test-containerize cycle
Dockerfile (Delmoment 2)	✓ Complete	Multi-stage build, tested with podman
docker-compose.yml (Delmoment 2)	✓ Complete	Webapp + CosmosDB emulator
Authentication (Delmoment 3)	✓ Complete	Cookie-based, two roles
Authorization (Delmoment 3)	✓ Complete	Role-based with [Authorize]
CosmosDB repository (Delmoment 3)	✓ Complete	Generic repository pattern
CI/CD pipeline (Delmoment 4)	✗ Not started	GitHub Actions workflow needed
Bicep IaC (Delmoment 4)	✗ Not started	Azure resource provisioning needed
Azure deployment (Delmoment 4)	✗ Not started	Container Apps + ACR needed
Verification (Delmoment 5)	✗ Not started	Depends on deployment

Known Issues

The CosmosDB emulator uses a self-signed certificate that the .NET SDK rejects by default. An `HttpClientFactory` with `DangerousAcceptAnyServerCertificateValidator` has been configured in `Program.cs` for the development environment, and `ConnectionMode.Gateway`

has been set. The emulator's `EMS_ENABLE_ENDPOINT_VALIDATION` environment variable has been set to `false`. These measures are intended to resolve the certificate validation issue, but the fix has not yet been fully verified in the containerized environment due to time constraints.

11. Conclusion

The core application logic, authentication, authorization, data layer, and containerization have been completed and tested locally. The CI/CD pipeline, Azure infrastructure provisioning, and deployment remain as future work. The application architecture follows a clean layered pattern (Domain → Data → Services → Web) and is ready for deployment once the Azure resources are provisioned and the pipeline is configured.